KLAi

Ship it: reproducible work, a small app and responsible use

How do we turn a notebook into something a colleague can repeat, use and trust?

AI and Data Science Professional | Session 23 of 24 | Module F: Ship it and present
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Make a notebook reproducible, save a model to a file and publish code and a small app for free



Write a model card and watch for data drift and concept drift



Run a fairness audit by group, spot a proxy variable, and name the main duties in the Uganda Data Protection and Privacy Act, 2019

Your two hours: concepts first, then a notebook lab

0:00	10	Recall and the question of the day
0:10	45	Concepts: picture, plain words, one example by hand
0:55	10	Break
1:05	45	Lab: predict, run, read, change
1:50	10	Exit check, keep and next

Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. A language model states a fact that is in none of your documents. What is that called, and what did we add to reduce it?

2. Why do we keep an evaluation set of test questions for a document assistant?

3. What is a chunk?



Answer aloud first. Then we check together.

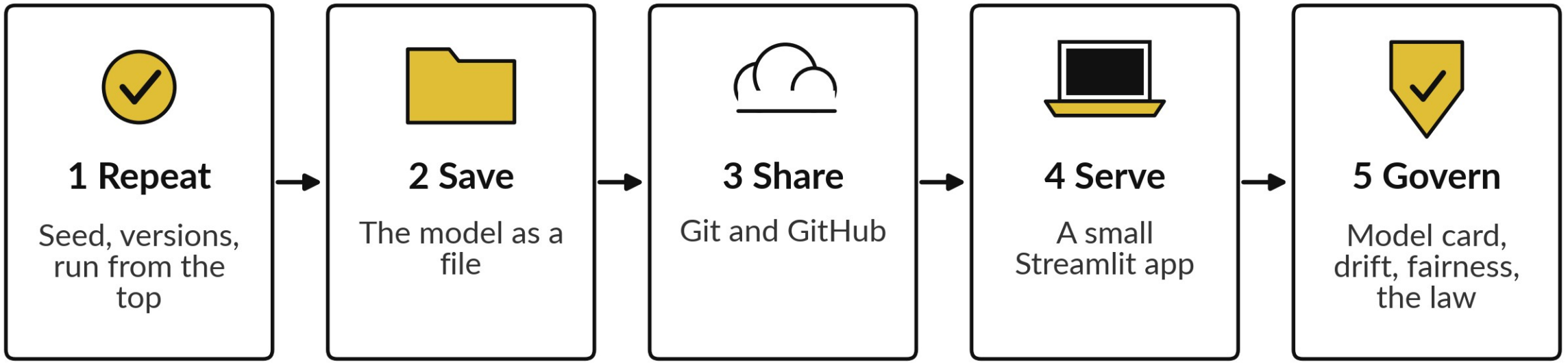
Answers to the three questions from Session 22

Question	Short answer
A language model states a fact that is in none of your documents. What is that called, and what did we add to reduce it?	A hallucination: fluent text that is invented. Retrieval-augmented generation (RAG) finds passages in your own documents first and answers from them.
Why do we keep an evaluation set of test questions for a document assistant?	So we can run the same questions after every change and see whether the answers got better or worse.
What is a chunk?	A piece of a document, such as one paragraph of a policy, that is stored and searched on its own.

Words we use today

	Word	What it means
	**Reproducibility**	A colleague who runs your notebook gets your result. It needs a fixed random seed, recorded library versions and a run from the top.
	**Model serialisation (persistence)**	Saving a trained model to a file, for example with joblib or pickle, so it can be loaded later without training again.
	**Data drift (and concept drift)**	Data drift: the inputs change after training. Concept drift: the link between the inputs and the outcome changes.
	**Fairness audit (proxy variable, disparate impact)**	Compare approval and error rates across groups. A proxy variable stands in for a column you left out. Disparate impact is the ratio of two approval rates.
	**Personal data (data subject, data controller)**	Information that identifies a person. The person is the data subject. The organisation that decides how it is used is the data controller.

Shipping a model means five jobs after the notebook works



Today covers all five. The lab does 1 and 2, writes the files for 4, and does 5. You do 3 and the deployment with your own free accounts.

A random seed makes a random draw repeatable

Five loans drawn at random from 8,000

seed 42, run 1	2215	2582	1662	3027	4343	
seed 42, run 2	2215	2582	1662	3027	4343	**identical**
seed 43, run 1	5762	1863	1717	2500	2475	**different**
seed 43, run 2	5762	1863	1717	2500	2475	

Each box is the row number of one loan in loans.csv.

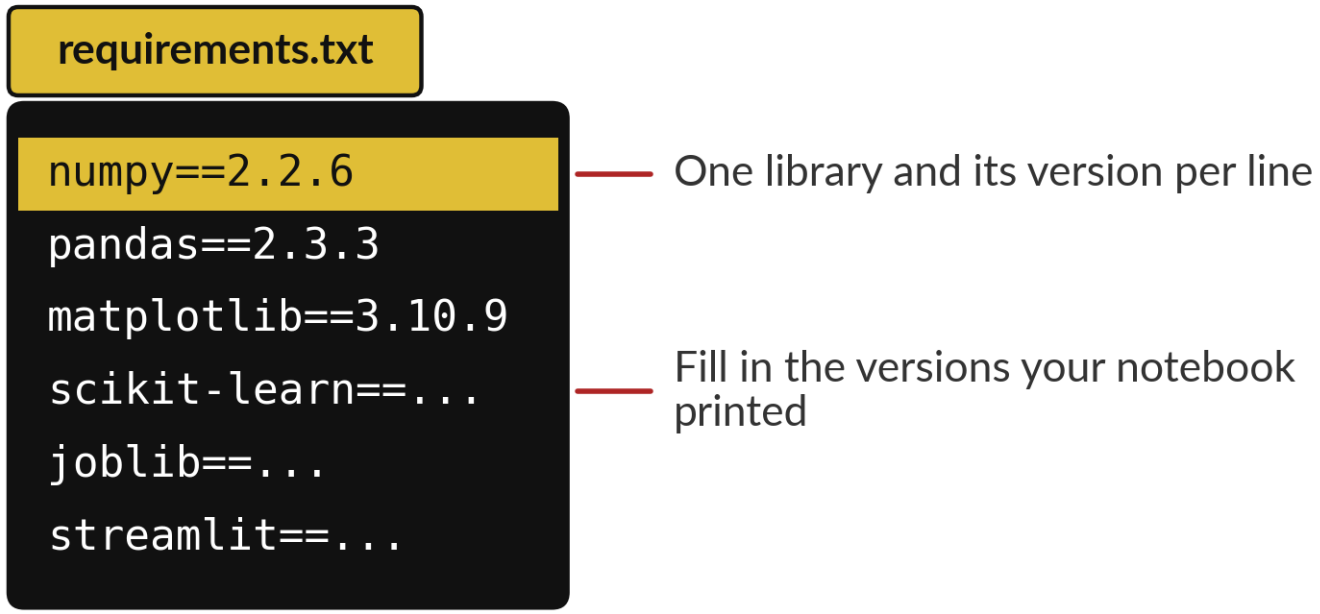
Seed (random_state): a starting number for the random generator. The same seed gives the same draw.

In pandas: random_state=42.

In Sheets, paste RAND as values.

Use one seed for every random split.

A requirements file records which library versions your notebook used



Requirements file (requirements.txt): a text file that lists each library and its version.

The versions shown come from one machine. Print your own: numpy.__version__, or pip freeze.

Keep only libraries you import.

Install with pip install -r requirements.txt.

Restart and run all is the test of a notebook

Works only for you

You ran cells in a different order.

Your open session remembers a deleted cell.

A colleague meets a NameError.

Works for anyone

Runtime, then Restart session and run all.

Fix the first error and run again.

Share when it runs top to bottom.

Run it from the top before you share it.

Model serialisation saves a trained model to a file you can load later



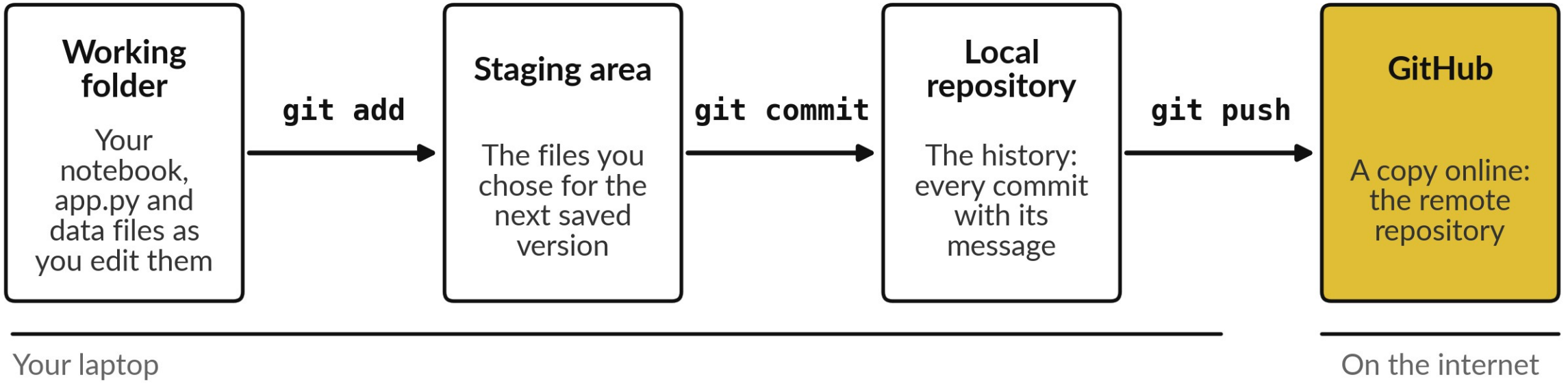
Model serialisation (persistence): saving a trained model to a file.

Tools: joblib or pickle. joblib uses pickle underneath.

Load only files you trust: loading can run hidden code.

Use the same library versions.

Git keeps a history of your project, and GitHub keeps a copy online



Git is the tool on your laptop. GitHub hosts a copy online.

Six words you will hear about Git and GitHub

	Word	Plain meaning	You will see
	**Repository (repo)**	The project folder together with its full history.	github.com/you/loan-model
	**Commit**	One saved version, with a message saying what changed.	git commit -m "first version"
	**Branch**	A separate line of work. The main line is called main.	main
	**Push**	Upload your commits to GitHub.	git push -u origin main
	**README**	The front page of a repository: what the project is and how to run it.	README.md
	**.gitignore**	A list of files Git must never track, such as data with personal details or passwords.	.gitignore

A public repository is public: personal data, passwords and keys never go in

Put in

The notebook, run from the top.

app.py and requirements.txt.

A README and the model card.

Generated or public data.

Never put in

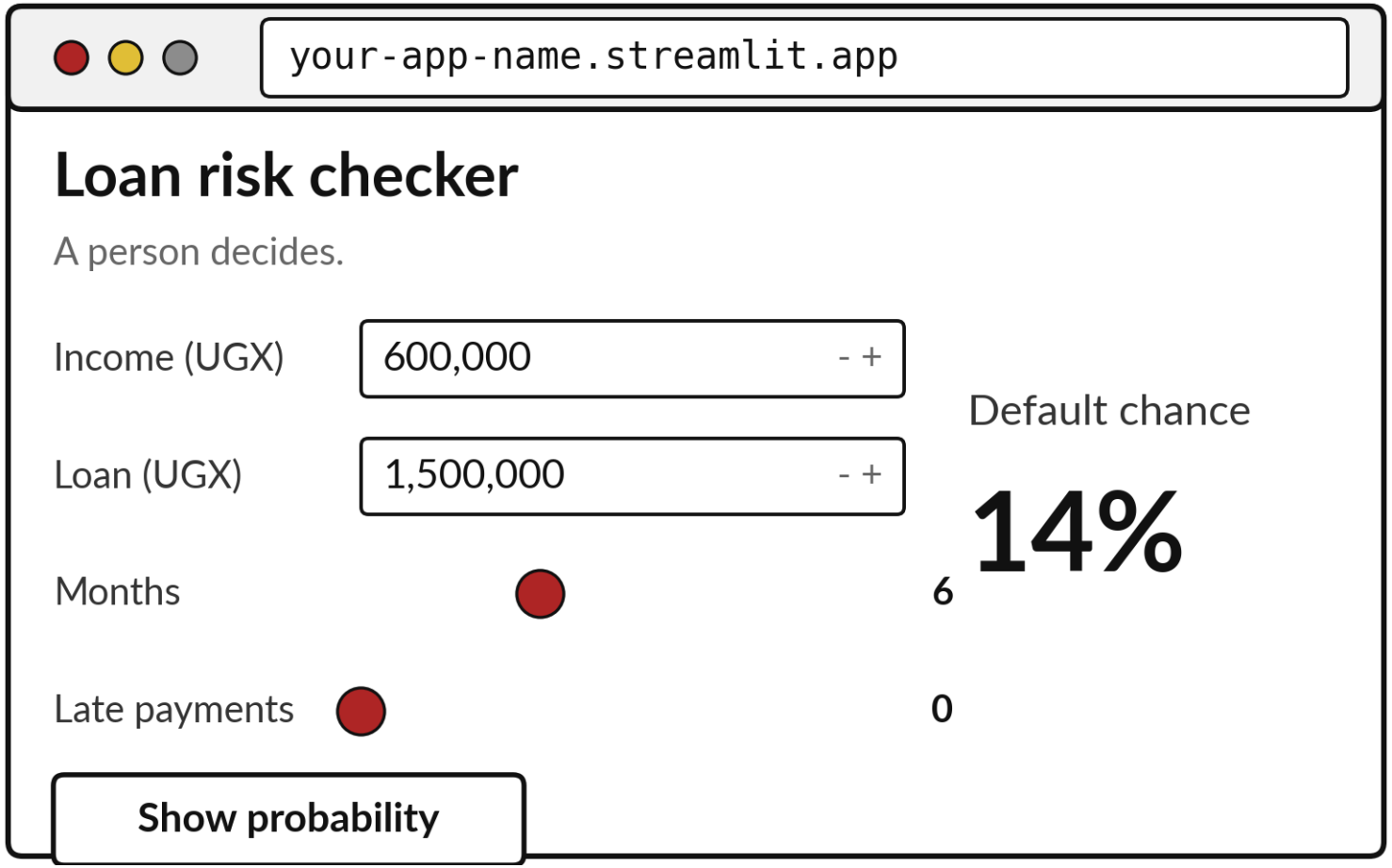
Names, phones or loan records of real people.

Passwords, keys and tokens.

Data your employer has not cleared.

Deleting a file later does not erase its history. Check before you push.

A Streamlit app turns a saved model into a web page



Streamlit: a free Python library that builds a web page from a script.

You write inputs and output. Streamlit draws the page.

These inputs give 14% chance of default.

A person makes the decision.

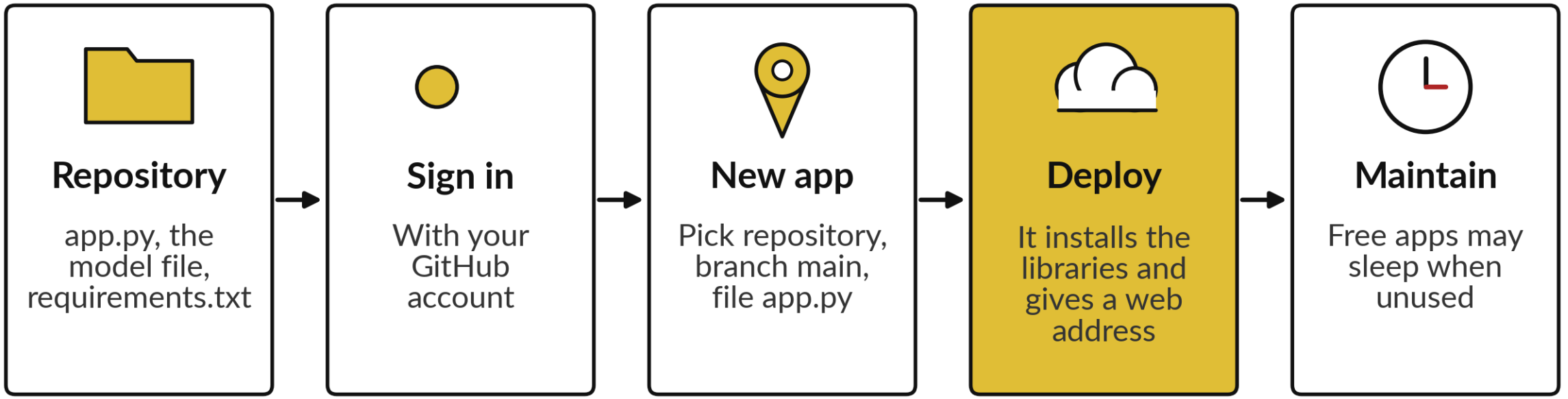
The whole app is about 25 lines: load the model, ask for four inputs, predict

```
 1  import joblib
 2  import pandas as pd
 3  import streamlit as st
 4
 5  model = joblib.load('loan_model.joblib')
 6
 7  st.title('Loan risk checker')
 8  st.caption('A person decides.')
 9
10  income = st.number_input('Income (UGX)',
11                           value=600000)
12  amount = st.number_input('Loan (UGX)',
13                           value=1500000)
```

```
14  months = st.slider('Months', 3, 12, 6)
15  late = st.slider('Late payments', 0, 6, 0)
16
17  row = pd.DataFrame({
18    'monthly_income_ugx': [income],
19    'loan_amount_ugx': [amount],
20    'term_months': [months],
21    'previous_late_payments': [late]})
22
23  if st.button('Show probability'):
24    p = model.predict_proba(row)[0, 1]
25    st.metric('Default chance', f'{p:.0%}')
```

Gold lines: load the model (line 5) and predict (line 24).

Streamlit Community Cloud can deploy your app from a GitHub repository



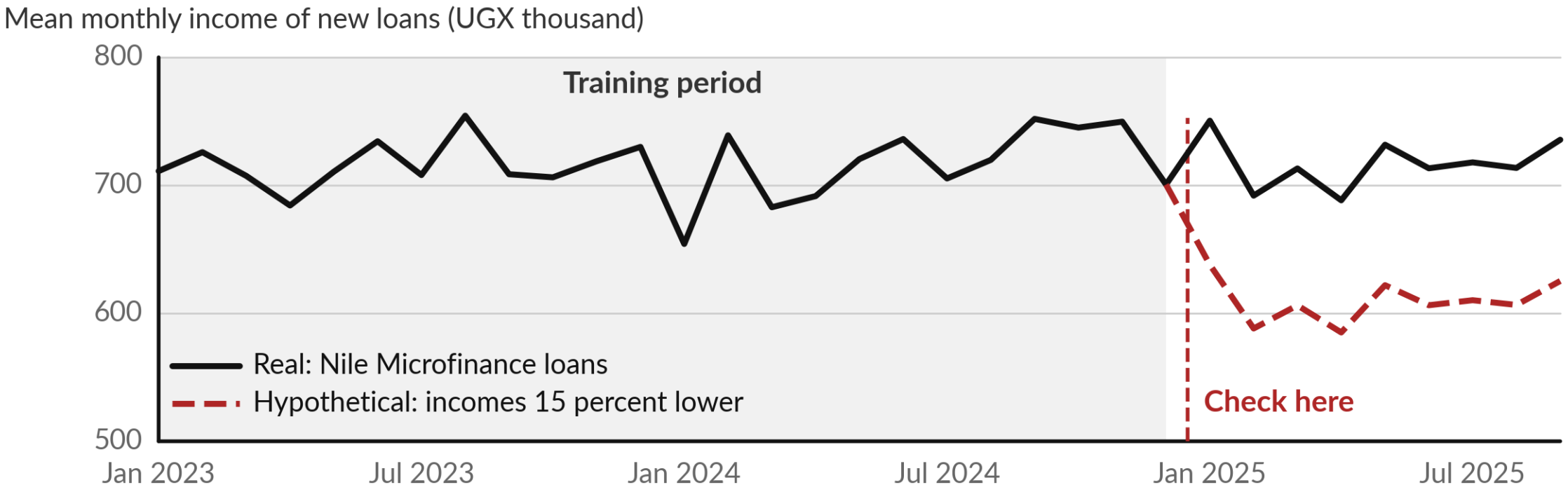
Deploy: put the app on a server. Free at the time of writing, with limits on resources and sleeping apps. Check current terms, including private repositories.

A model card is a one-page note on what a model is for and where it can fail

Section	The loan model (an illustrative card)
Purpose	Ranks loan applications so an officer knows which to review first. It does not approve or refuse.
Data	8,000 fictional loans, 2023-01 to 2025-09. collection_calls was removed because it leaks the answer.
Metrics	Simplified four-variable model. AUC 0.727 on 2,000 held-out loans. At decision threshold 0.15 it flags 28% of loans and finds 59% of defaults.
Limits	Fictional data. 379 loans lack income and are filled with the median. Four variables only. Not tested after the data changes.
Fairness and review	On the held-out loans, approval women 72.3%, men 71.6%. Reviewed by the credit manager and the data protection officer.

Update the card every time the model changes.

Data drift: when new data stops looking like the training data, the model gets worse



Solid: real. Dashed: a hypothetical 15 percent fall from month 25.

Data drift and concept drift are different problems

Data drift

The inputs change: incomes or loan sizes move.

Check: compare each month of inputs with the training period.

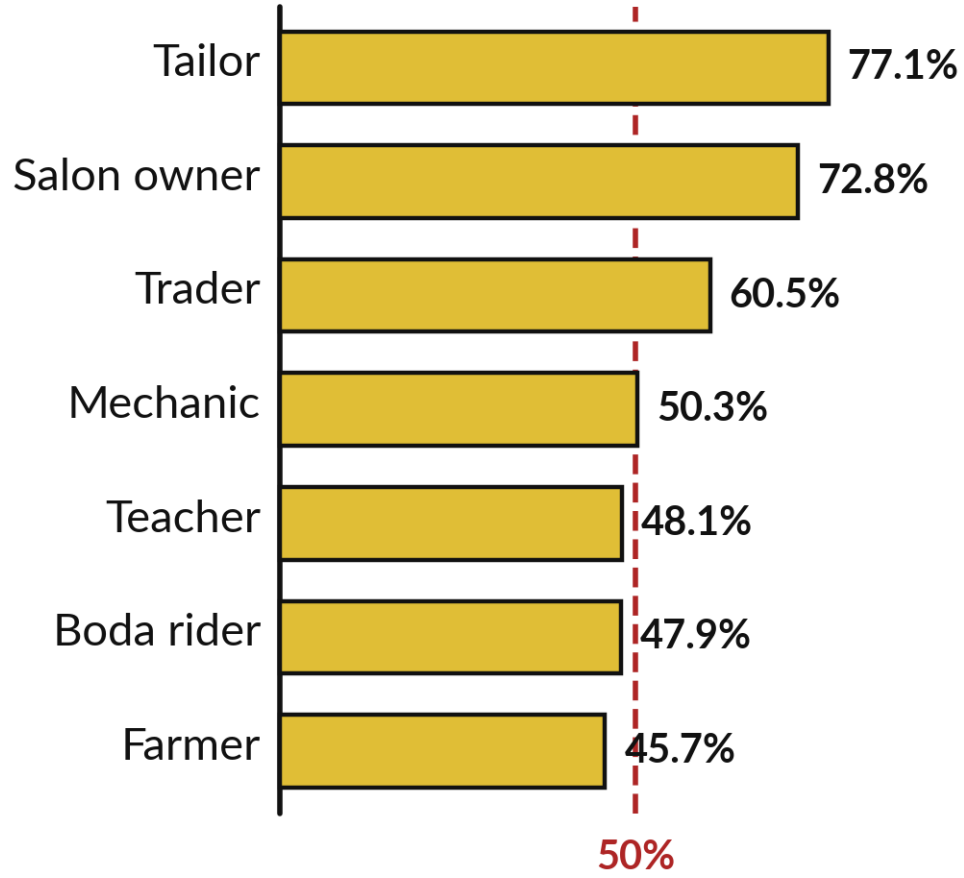
Concept drift

The link between inputs and outcome changes. Illustration: a new repayment option changes the risk.

Check: compare predicted with actual defaults.

Plan monitoring, set a review date, retrain when needed.

A proxy variable carries the information of a column you removed



Proxy variable: a variable that stands in for another. Occupation stands in for gender.

Women are 77% of tailors and 73% of salon owners.

Dropping the gender column keeps the information.

Occupation rules can treat men and women differently.

Disparate impact: divide the lower approval rate by the higher one

Approval rate

$$\text{approval rate} = \text{loans not flagged} \div \text{loans in the group}$$

Disparate impact ratio

$$\text{DI} = \frac{\text{lower approval rate}}{\text{higher approval rate}}$$

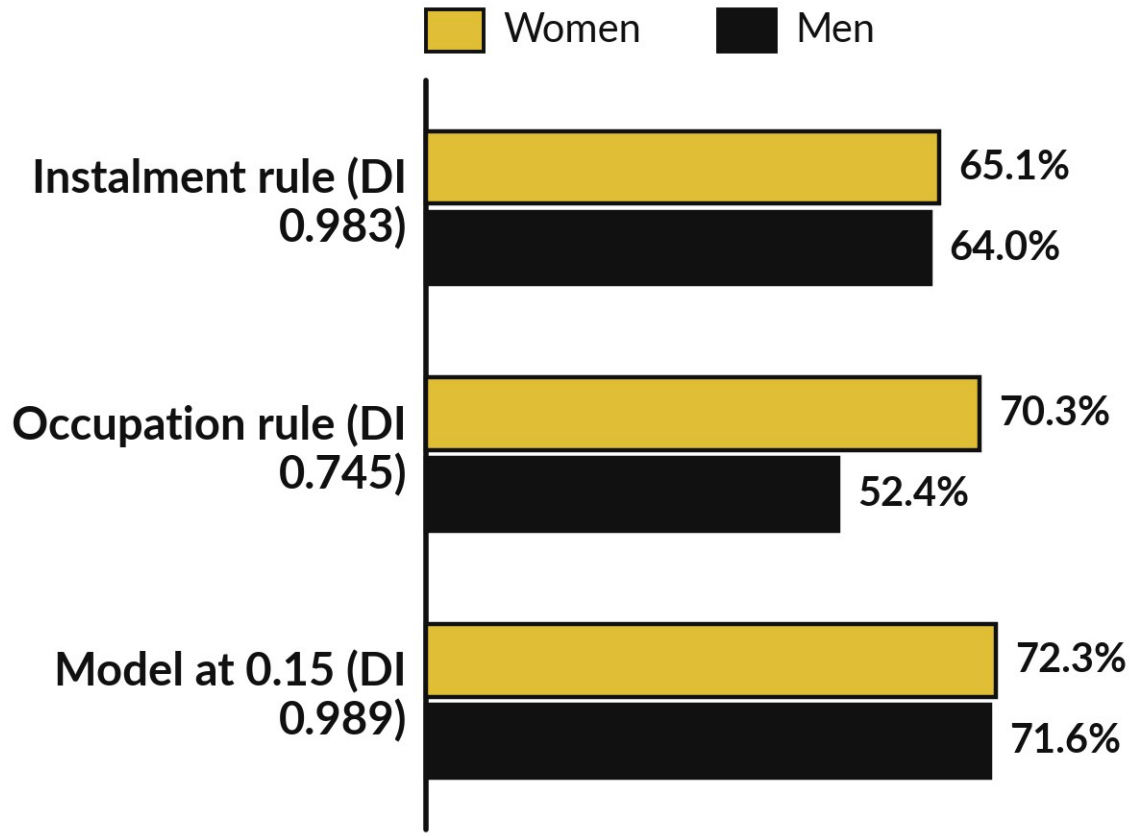
Occupation rule, women and men

$$\text{DI} = 0.524 \div 0.703 = 0.745$$

Symbol	What it means	In our data
Approval rate	Share of a group not flagged as high risk.	women 70.3%, men 52.4%
DI	Disparate impact ratio. 1.00 means equal rates.	0.745 for the occupation rule
0.8 rule of thumb	Below 0.8 is a warning. It comes from US employment practice and is no Ugandan legal test.	0.745 is below 0.8

A screening number: it starts a question.

A fairness audit compares approval rates by group, and one rule raises a flag



Fairness audit: compare approval and error rates across groups.

Approval rate: share of loans not flagged.

The occupation rule favours women.

The other two treat groups alike.

Error rates differ too: the occupation rule misses more defaults among women

Rule and group	Approval rate	Defaults missed (false negative rate)	Repaid loans flagged (false positive rate)
Instalment rule, women	65.1%	32.8%	30.5%
Instalment rule, men	64.0%	37.5%	32.0%
Occupation rule, women	70.3%	65.7%	29.1%
Occupation rule, men	52.4%	42.3%	46.1%

False negative rate: share of actual defaults the rule did not flag. False positive rate: share of repaid loans it flagged.

Uganda's Data Protection and Privacy Act, 2019: the key roles (a teaching summary)

	Term	Plain meaning	In a loan file
	**Personal data**	Information from which a person can be identified.	Name, phone number, loan record
	**Data subject**	The person the data is about.	The borrower
	**Data controller**	The organisation that decides why and how personal data is used.	The lender
	**Data processor**	Someone who handles data for the controller.	A cloud or software provider
	**Regulator**	My understanding: the Personal Data Protection Office, under NITA-U.	Where complaints go

Hedged summary from public descriptions. Not legal advice. Read the Act, or ask your data protection officer or a lawyer.

Public summaries of the Act describe seven principles: collect less, say why, keep it safe

Principle	In plain words
Accountability	Someone is responsible and can show what was done.
Fair and lawful processing	Use data fairly and within the law.
Adequate and relevant data	Collect only what the purpose needs.
Kept only as long as needed	Delete data when the purpose ends.
Quality of information	Keep data accurate and up to date.
Openness and participation	Tell people what you hold and why.
Safety and security	Protect data from loss, leaks and misuse.

Teaching summary of public descriptions. Not legal advice. Read the Act for the exact wording.

Data subjects have rights, and a model team has matching jobs

Right	In plain words	A model team keeps
Access	Ask what data an organisation holds about you.	A record of which data feeds the model
Correction	Ask for wrong data to be fixed.	A way to correct a record and retrain
Erasure	Ask for data to be deleted, in the cases the law allows.	A deletion process that covers copies
Automated decisions	Ask not to be subject to a decision taken only by a computer.	A person who reviews the model output
Consent and cloud tools	My understanding: the Act expects a lawful basis such as consent. Personal data sent outside Uganda may need safeguards.	No real personal data in Colab, GitHub or an app host until checked

My understanding from public summaries. Check the Act and take advice for your own case.

Where you will use this at work: make it repeatable, then make it checkable

	Your job	What you ship	What today lets you do
	**Credit analyst at a bank or SACCO, using Python and scikit-learn**	A default-risk model	Save it with joblib, write a model card, and audit approval and error rates by gender and district before release.
	**Officer in a ministry, using pandas and Excel**	A monthly report notebook	Fix the seed, record the versions, and rerun it each month from the top so the numbers reproduce.
	**Sales analyst at a retailer, using Streamlit**	A forecast tool for branch managers	Wrap the model in a small app, with a README, in a repository that holds no customer data.
	**Data officer at an NGO, using GitHub**	A survey analysis	Keep the code in Git, keep personal data out, and check the duties of the data controller.

Read the recipe before the lab: a seed and an approval rate by group

```
known = test.dropna(subset=['monthly_income_ugx'])
pay = known.loan_amount_ugx * 1.15 / known.term_months
ratio = pay / known.monthly_income_ugx
known = known.assign(flag=(ratio > 0.40).astype(int))
approval = 1 - known.groupby('gender').flag.mean()
print(approval.round(3).to_dict())
print(round(approval.min() / approval.max(), 3))
```

```
{'F': 0.651, 'M': 0.64}
0.983
```

In plain words

Line 1 keeps the held-out (test) loans that have an income.

Lines 2 to 4 flag a loan when the ratio (instalment_to_income) is above 0.40. The 1.15 is an illustrative charge factor.

Line 5 is the approval rate: one minus the flag rate, for each gender.

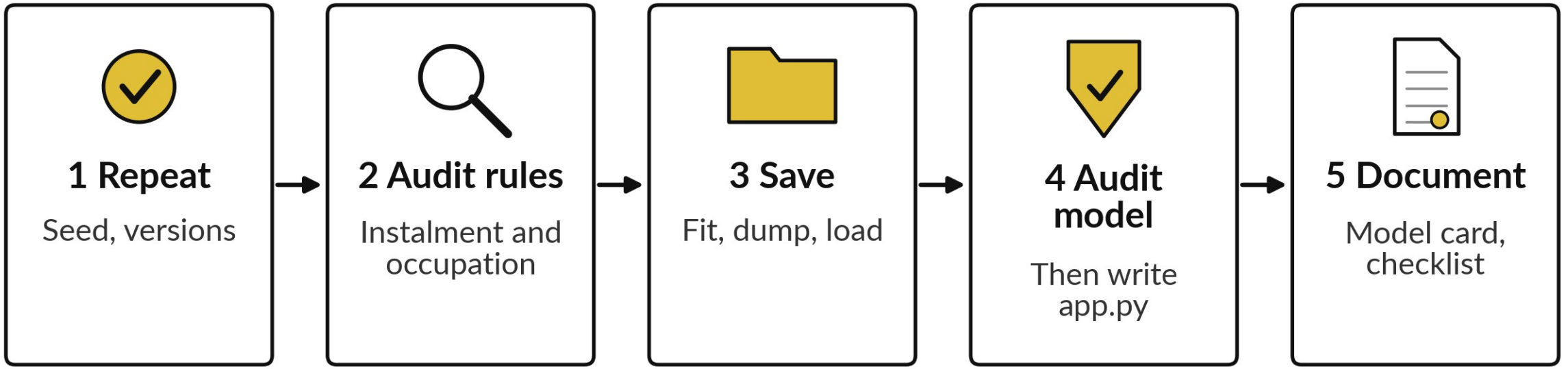
Line 7 is the disparate impact ratio: 0.983.



Part 2: the notebook lab

Close the slides. Open the notebook.

The lab has five parts, and you keep a folder ready to push



Twenty-seven checks in 45 minutes. In each step: predict, run, read, change one thing.

Check your numbers before you move on

	What you compute	You should get
	Row numbers drawn with random_state=42	2215, 2582, 1662, 3027, 4343
	Default rate: women, and Masaka	12.1% and 14.8%
	Instalment rule (held-out loans): approval women, men, and DI	65.1%, 64.0%, 0.983
	Occupation rule (held-out loans): approval women, men, and DI	70.3%, 52.4%, 0.745
	Occupation rule: defaults missed, women and men	65.7% and 42.3%
	Model: held-out AUC, and first-loan probability	0.727 and 0.22
	Model at 0.15: approval women, men	72.3% and 71.6%
	Lines in app.py, and libraries in requirements.txt	25 and 5

The model numbers may differ in the third decimal because the solver differs.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
NameError: name is not defined	A cell above did not run, or you ran cells out of order.	Runtime, Restart session and run all. Fix the first error.
Different row numbers from the table	The seed was not set, or you used a different seed.	Use random_state=42 in both draws.
Disparate impact above 1	You divided the higher rate by the lower one.	Divide the lower approval rate by the higher one.
ModuleNotFoundError for joblib	The library is not installed in this session.	Run pip install joblib in a cell, then rerun.
Model probabilities all about the same	You skipped the imputer or scaler in the pipeline.	Use make_pipeline with the imputer, scaler and model.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**Name three habits that let a colleague repeat your notebook.**	Set the random seed, record the library versions, and restart and run all from the top.
	**A rule never sees gender, yet approves 70.3% of women and 52.4% of men. What is going on, and what next?**	A proxy variable (occupation) carries the information. Report the disparate impact and error rates by group, and justify or change the rule.
	**In a loan file, who is the data subject and who is the data controller?**	The borrower is the data subject. The lender, which decides how the data is used, is the data controller.

Save your shipping folder and push it to your own GitHub

What you keep from today

Keep your shipping folder: the notebook, requirements.txt, loan_model.joblib, app.py and a completed model card with a fairness audit and a data protection checklist.

Our question today

How do we turn a notebook into something a colleague can repeat, use and trust?

Next session

Session 24: Present your course project and plan your path from here. You tell a result as an answer, evidence and limits, and plan what to learn next.

Take away

Your lab folder

Ready to push

The audit

Rates by group

Next session

Present it